Reber Grammar

DFA with associative transition memory

Reber's grammar (1967) is a small finite-state grammar over the letters T, P, S, X and V. Strings start with B and end with E. In artificial grammar learning experiments, participants who memorized strings generated by the grammar could later tell grammatical from ungrammatical strings, without being able to state the rules. The grammar has since become a standard benchmark for sequence learning in neural networks. Every letter labels two different transitions, so the next letter depends on the current state, not just on the letter before it.

This notebook implements the grammar as a finite automaton in a "transition" memory component. The query is a bundle of the current state and the current letter, and the memory returns the next state. The state is fed back and bundled with the next letter. The grammar is cyclic: every string starts and ends in the state "end", which is presented together with the first letter of each string.

Each of the twelve rules is taught once. The automaton then classifies grammatical strings generated by the grammar and ungrammatical strings derived from other grammatical strings by substituting a single letter. A string is accepted if the automaton reaches the state "end" after its last letter. An undefined transition returns the empty set, and the automaton stays halted for the rest of the string.

Requirements

```
<< "Circuits.m"
```

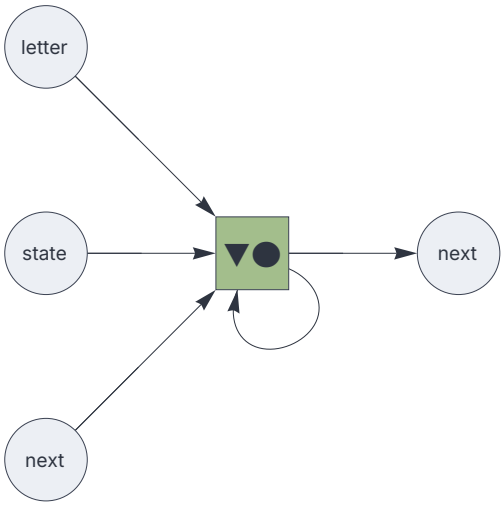
Circuit

```
jsonstr = Import[NotebookDirectory[] <> "ReberGrammar.json", "String"];
```

```
jsonstr =
```

```
{ "hyperparameters": {"default": [2000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [1], "label": "next"},
    {"component": "input", "plugin": "codec", "send": [2], "label": "state"},
    {"component": "input", "plugin": "codec", "send": [3], "label": "letter"},
    {"component": "transition", "send": [11], "threshold": 1.5,
        "receive": [1, [2, 11, 3]]},
    {"component": "output", "plugin": "codec", "receive": [11], "label": "state"}
  ]}
```

```
circ = Circuit[jsonstr];
f = circ["function"];
circ["schematics"][ImageSize→270]
```



Grammar

```
datadir = NotebookDirectory[] <> "../../data/reber-grammar/";
```

```
grammar = Rest[Import[datadir <> "grammar.csv"]];
```

```
grammar // TableForm
```

```
end    B    0
0      T    1
0      P    2
1      S    1
1      X    3
2      T    2
2      V    4
3      X    2
3      S    5
4      P    3
4      V    5
5      E    end
```

Train

Each rule once. An empty cycle follows each rule, so the next rule starts without a fed-back state.

```
Scan[(f[#[[3]], #[[1]], #[[2]]]; f[{}, {}, {}]) &, grammar];
```

Datasets

```
n = 1000;
valid = Take[Import[datadir <> "reber.txt", "Lines"], -n];
invalid = Take[Import[datadir <> "non-reber.txt", "Lines"], n];
```

Test setup

Run the automaton: the first letter comes with the start state “end”, the remaining letters arrive alone.

```
trace[s_String] := Module[{letters = Characters[s], first},
  first = f[{}, "end", First[letters]];
  Prepend[f[{}, {}, #] & /@ Rest[letters], first]];
acceptedQ[s_String] := Last[trace[s]] === "end";
```

Run

```
Do[Print[s, "   ",
    StringRiffle[MapThread[#1 <> ":" <> ToString[Replace[#2, Null → "-"]] &,
      {Characters[s], trace[s]}], " "],
    "   ", If[acceptedQ[s], "accepted", "rejected"]],
  {s, Join[Take[valid, 3], Take[invalid, 3]]}];

(* Grammatical and ungrammatical strings alternate, so that every
   rejected string is followed by a string that must be accepted *)
results = Table[{acceptedQ[valid[[i]]], acceptedQ[invalid[[i]]]}, {i, n}];

Print["grammatical accepted:   ", Count[results[[All, 1]], True], "/", n];
Print["ungrammatical rejected: ", Count[results[[All, 2]], False], "/", n];
```

```
BTSSXSE   B:0 T:1 S:1 S:1 X:3 S:5 E:end   accepted
BPVVE   B:0 P:2 V:4 V:5 E:end   accepted
BPTVVE   B:0 P:2 T:2 V:4 V:5 E:end   accepted
BPVTSE   B:0 P:2 V:4 T:- S:- E:-   rejected
BTSSE   B:0 T:1 S:1 S:1 E:-   rejected
BTTVPSE   B:0 T:1 T:- V:- P:- S:- E:-   rejected
grammatical accepted:   1000/1000
ungrammatical rejected: 1000/1000
```